# Transport Layer Security

Module VII

# Transport Layer Security (TLS)

- The Transport Layer Security (TLS) protocol is the IETF standard version of the SSL protocol. The two are very similar, with slight differences.

**Differences between TLS and SSL**

Version. The first difference is the version number (major and minor). The current version of SSL is 3.0; the current version of TLS is 1.0. In other words, SSLv3.0 is compatible with TLSv1.0.

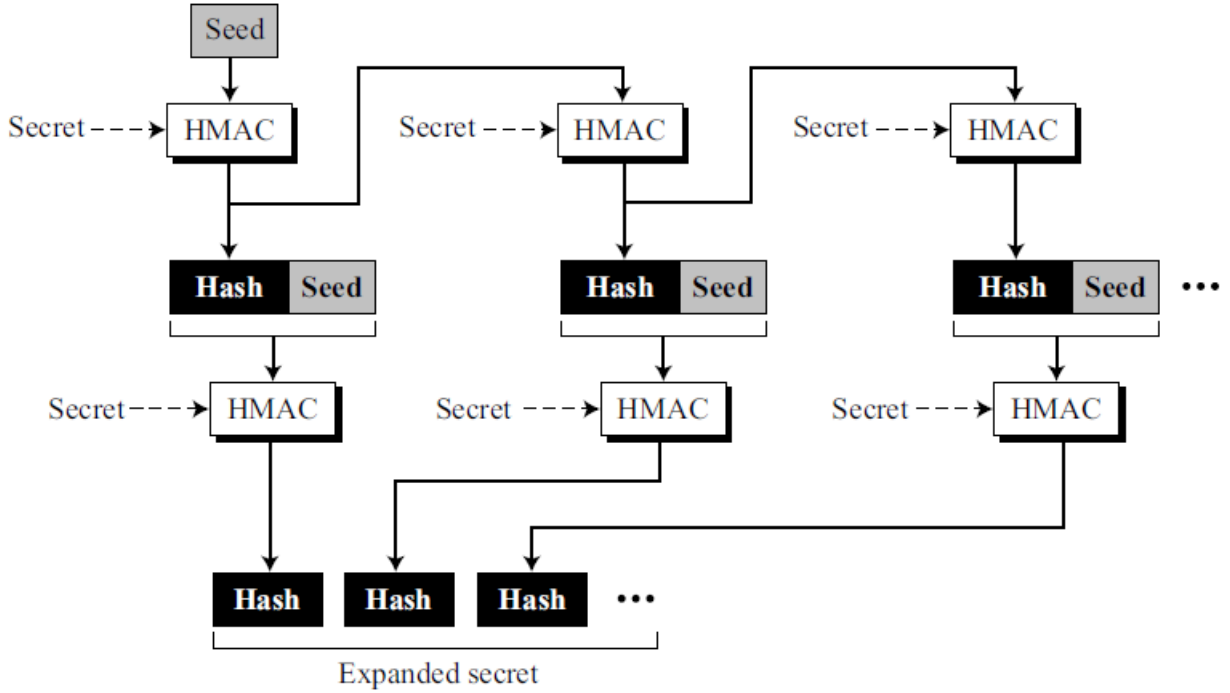
Cipher Suite. Another minor difference between SSL and TLS is the lack of support for the Fortezza method. TLS does not support Fortezza for key exchange or for encryption/decryption.

Generation of Cryptographic Secrets. The generation of cryptographic secrets is more complex in TLS than in SSL. TLS first defines two functions: the data-expansion function and the pseudorandom function.

## Generation of Cryptographic Secrets

# Data-expansion function

- It uses a predefined HMAC (either MD5 or SHA-1) to expand a secret into a longer one. This function can be considered a multiple section function, where each section creates one hash value.
- The extended secret is the concatenation of the hash values. Each section uses two HMACs, a secret and a seed. The data-expansion function is the chaining of as many sections as required.
- However, to make the next section dependent on the previous, the second seed is actually the output of the first HMAC of the previous section as shown in the figure.

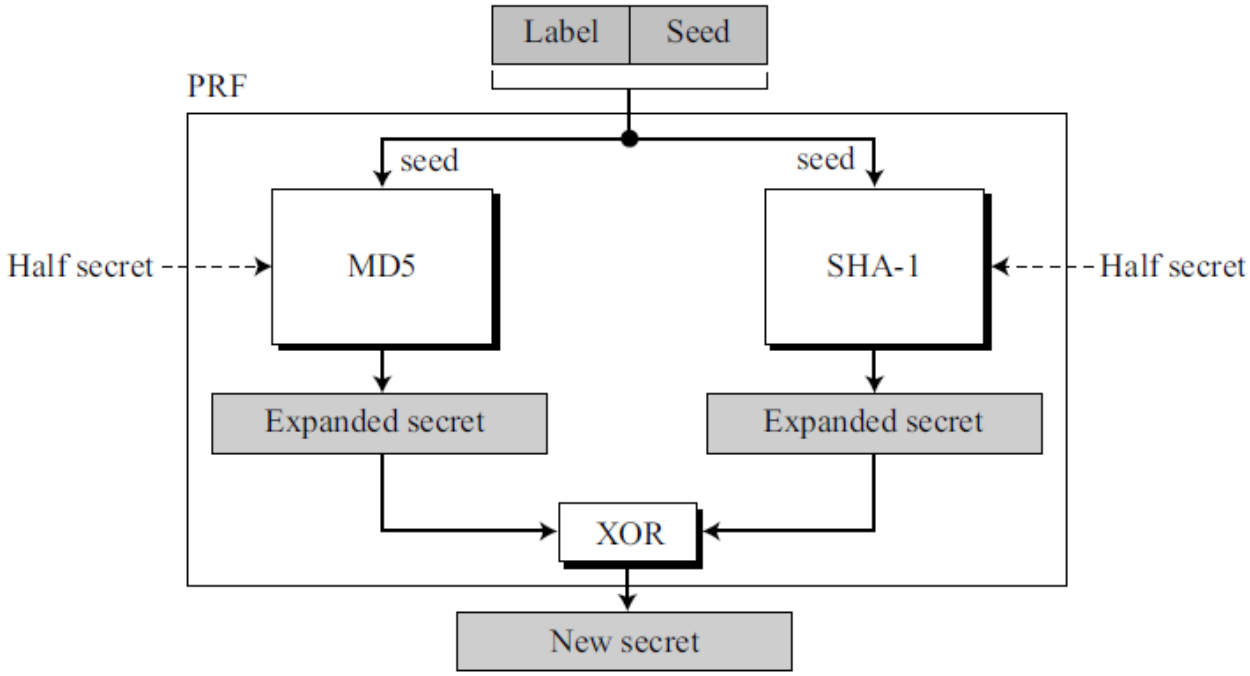



**Data-expansion function**

## Generation of Cryptographic Secrets

# Pseudorandom Function (PRF)

- TLS defines a pseudorandom function (PRF) to be the combination of two data-expansion functions, one using MD5 and the other SHA-1. PRF takes three inputs, a secret, a label, and a seed.
- The label and seed are concatenated and serve as the seed for each dataexpansion function. The secret is divided into two halves; each half is used as the secret for each data-expansion function. The output of two data-expansion functions is exclusive-ored together to create the final expanded secret.
- Since the hashes created from MD5 and SHA-1 are of different sizes, extra sections of MD5-based functions must be created to make the two outputs the same size. Figure shows the idea of PRF.



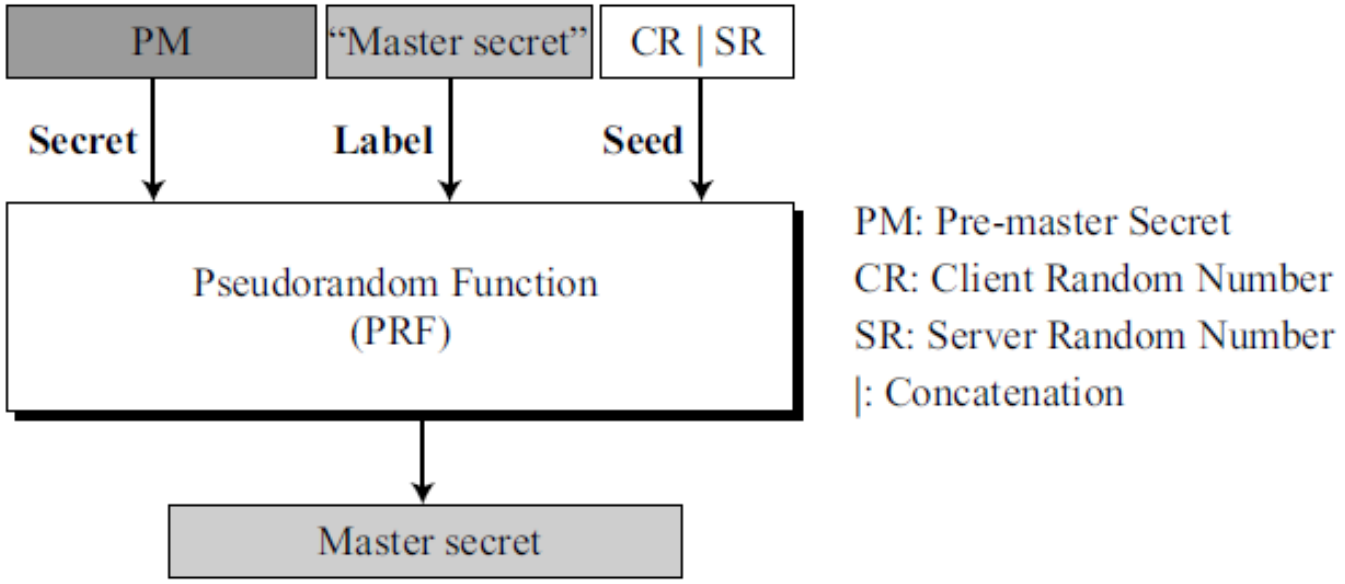

**PRF**

# Cipher Suite for Transport Layer Security (TLS)

| *Cipher suite* | *Key Exchange* | *Encryption* | *Hash* |
|---|---|---|---|
| TLS_NULL_WITH_NULL_NULL | NULL | NULL | NULL |
| TLS_RSA_WITH_NULL_MD5 | RSA | NULL | MD5 |
| TLS_RSA_WITH_NULL_SHA | RSA | NULL | SHA-1 |
| TLS_RSA_WITH_RC4_128_MD5 | RSA | RC4 | MD5 |
| TLS_RSA_WITH_RC4_128_SHA | RSA | RC4 | SHA-1 |
| TLS_RSA_WITH_IDEA_CBC_SHA | RSA | IDEA | SHA-1 |
| TLS_RSA_WITH_DES_CBC_SHA | RSA | DES | SHA-1 |
| TLS_RSA_WITH_3DES_EDE_CBC_SHA | RSA | 3DES | SHA-1 |
| TLS_DH_anon_WITH_RC4_128_MD5 | DH_anon | RC4 | MD5 |
| TLS_DH_anon_WITH_DES_CBC_SHA | DH_anon | DES | SHA-1 |
| TLS_DH_anon_WITH_3DES_EDE_CBC_SHA | DH_anon | 3DES | SHA-1 |
| TLS_DHE_RSA_WITH_DES_CBC_SHA | DHE_RSA | DES | SHA-1 |
| TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA | DHE_RSA | 3DES | SHA-1 |
| TLS_DHE_DSS_WITH_DES_CBC_SHA | DHE_DSS | DES | SHA-1 |
| TLS_DHE_DSS_WITH_3DES_EDE_CBC_SHA | DHE_DSS | 3DES | SHA-1 |
| TLS_DH_RSA_WITH_DES_CBC_SHA | DH_RSA | DES | SHA-1 |
| TLS_DH_RSA_WITH_3DES_EDE_CBC_SHA | DH_RSA | 3DES | SHA-1 |
| TLS_DH_DSS_WITH_DES_CBC_SHA | DH_DSS | DES | SHA-1 |
| TLS_DH_DSS_WITH_3DES_EDE_CBC_SHA | DH_DSS | 3DES | SHA-1 |

# Generation of Cryptographic Secrets

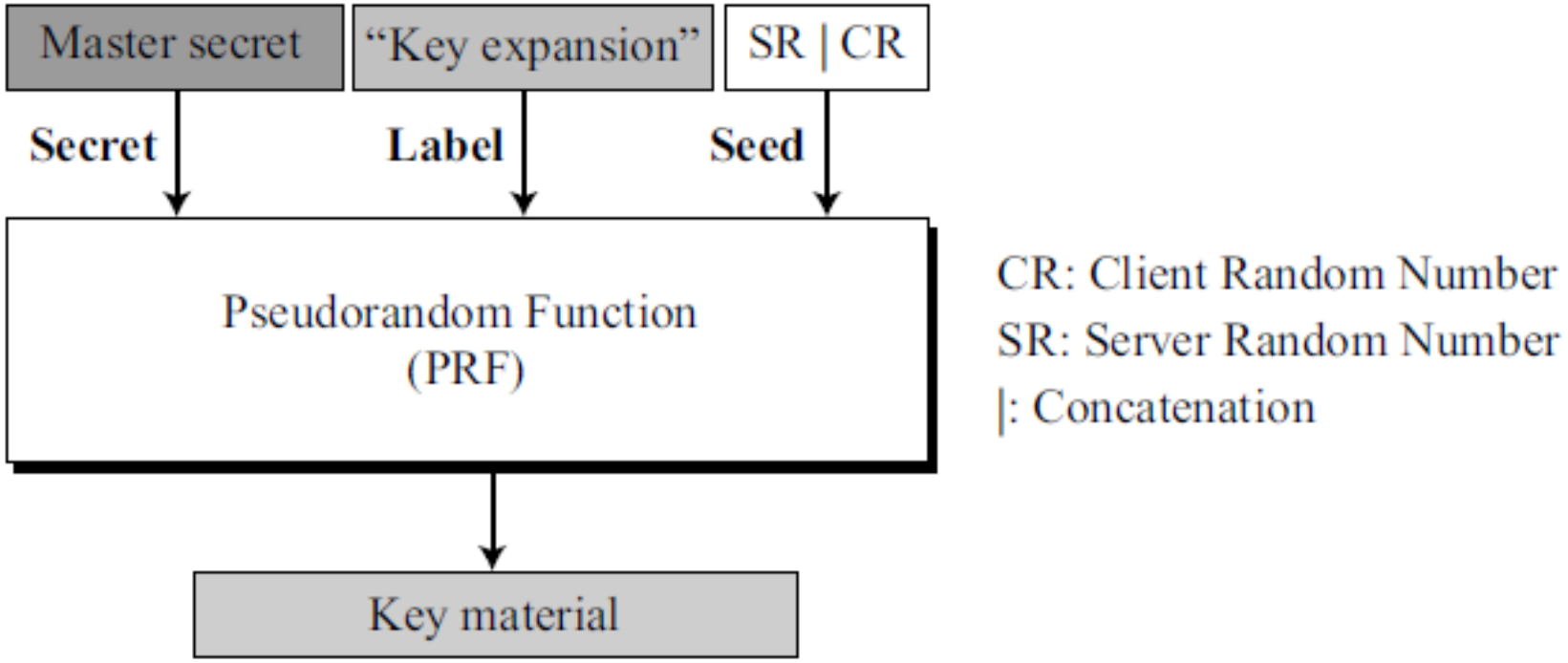
- **Premaster key.** The generation of the pre-master secret in TLS is exactly the same as in SSL.
- **Master key.** TLS uses the PRF function to create the master secret from the pre-master secret. This is achieved by using the pre-master secret as the secret, the string "master secret" as the label, and concatenation of the client random number and server random number as the seed. Note that the label is actually the ASCII code of the string "master secret". In other words, the label defines the output we want to create, the master secret. Figure shows the idea.




Master key generation

# Generation of Cryptographic Secrets

- **Key material**. TLS uses the PRF function to create the key material from the master secret. This time the secret is the master secret, the label is the string "key expansion", and the seed is the concatenation of the server random number and the client random number as shown in the figure.




**Key material generation**

# Alert protocol

- TLS supports all of the alerts defined in SSL except for NoCertificate. TLS also adds some new ones to the list. Table shows the full list of alerts supported by TLS.

Alerts defined for TLS

| *Value* | *Description* | *Meaning* |
|---|---|---|
| 0 | *CloseNotify* | Sender will not send any more messages. |
| 10 | *UnexpectedMessage* | An inappropriate message received. |
| 20 | *BadRecordMAC* | An incorrect MAC received. |
| 21 | *DecryptionFailed* | Decrypted message is invalid. |
| 22 | *RecordOverflow* | Message size is more than $2^{14} + 2048$. |
| 30 | *DecompressionFailure* | Unable to decompress appropriately. |
| 40 | *HandshakeFailure* | Sender unable to finalize the handshake. |
| 42 | *BadCertificate* | Received certificate corrupted. |
| 43 | *UnsupportedCertificate* | Type of received certificate is not supported. |
| 44 | *CertificateRevoked* | Signer has revoked the certificate. |
| 45 | *CertificateExpired* | Certificate has expired. |
| 46 | *CertificateUnknown* | Certificate unknown. |
| 47 | *IllegalParameter* | A field out of range or inconsistent with others. |
| 48 | *UnknownCA* | CA could not be identified. |

# Alert protocol cntd..

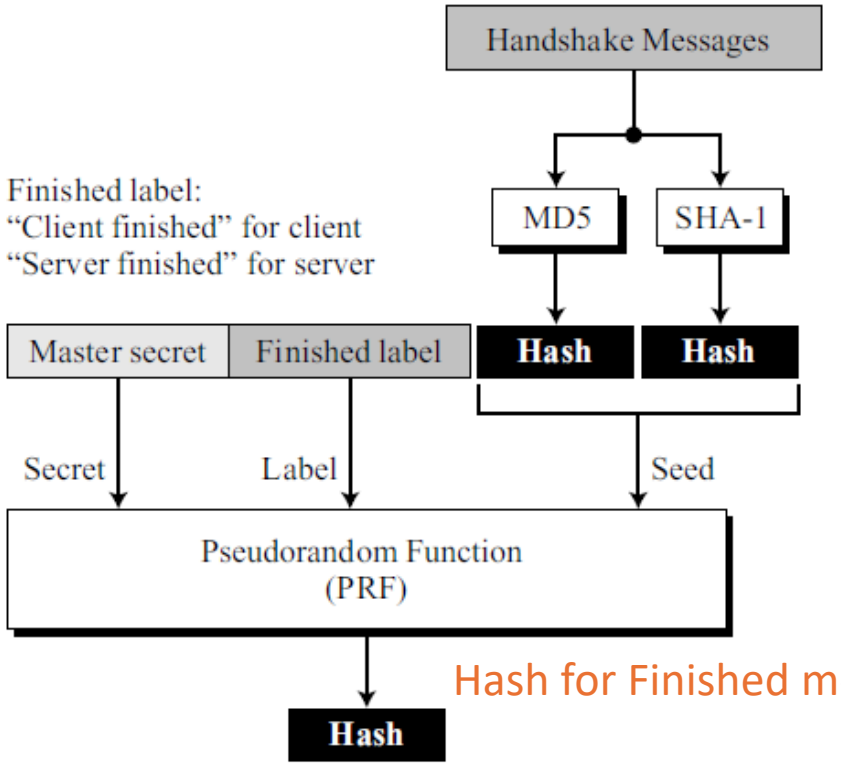
Alerts defined for TLS

| *Value* | *Description* | *Meaning* |
|---|---|---|
| 49 | *AccessDenied* | No desire to continue with negotiation. |
| 50 | *DecodeError* | Received message could not be decoded. |
| 51 | *DecryptError* | Decrypted ciphertext is invalid. |
| 60 | *ExportRestriction* | Problem with U.S. restriction compliance. |
| 70 | *ProtocolVersion* | The protocol version is not supported. |
| 71 | *InsufficientSecurity* | More secure cipher suite needed. |
| 80 | *InternalError* | Local error. |
| 90 | *UserCanceled* | The party wishes to cancel the negotiation. |
| 100 | *NoRenegotiation* | The server cannot renegotiate the handshake. |

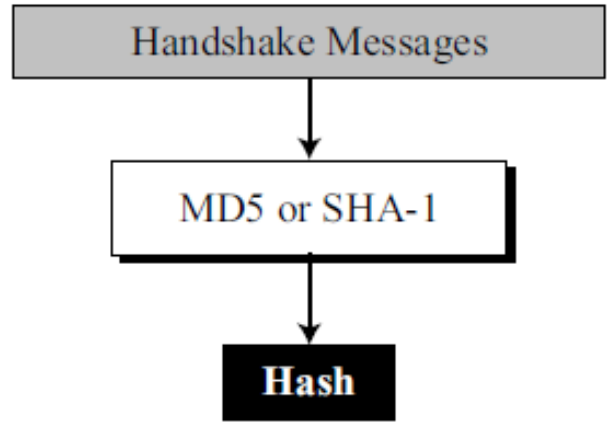
# Handshake protocol

- TLS has made some changes in the Handshake Protocol. Specifically, the details of the CertificateVerify message and the Finished message have been changed.
- CertificateVerify Message. In SSL, the hash used in the CertificateVerify message is the two-step hash of the handshake messages plus a pad and the master secret. TLS has simplified the process. The hash in the TLS is only over the handshake messages as shown in the figure.

Finished message. The calculation of the hash for the Finished message has also been changed. TLS uses the PRF to calculate two hashes used for the finished message as shown in the figure.




Hash for Finished message in TLS




*Hash for CertificateVerify message in TLS*

# Record Protocol

- The only change in the Record Protocol is the use of HMAC for signing the message. TLS uses the MAC to create HMAC. TLS also adds the protocol version (called Compressed version) to the text to be signed. Figure shows how the HMAC is formed.